Analyzing RCT with Precision by Adjusting for Baseline Covariates

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
```

Jonathan Roth's DGP

Here we set up a DGP with heterogenous effects. In this example, with is due to Jonathan Roth, we have

$$E[Y(0)|Z] = -Z, \quad E[Y(1)|Z] = Z, \quad Z \sim N(0,1).$$

The CATE is

$$E[Y(1) - Y(0)|Z] = 2Z.$$

and the ATE is

$$2EZ = 0.$$

We would like to estimate ATE as precisely as possible.

An economic motivation for this example could be provided as follows: Let D be the treatment of going to college, and let Z be academic skills. Suppose that academic skills cause lower earnings Y(0) in jobs that don't require a college degree, and cause higher earnings Y(1) in jobs that require college degrees. This type of scenario is reflected in the DGP set-up above.

```
def gen_data(random_seed):
    np.random.seed(random_seed)
    n = 1000             # sample size
    Z = np.random.normal(size=n)         # generate Z
    Y0 = -Z + np.random.normal(0, 0.1, size=n)   # conditional average baseline response is -Z
    Y1 = Z + np.random.normal(0, 0.1, size=n)    # conditional average treatment effect is +Z
    D = np.random.binomial(1, .2, size=n)    # treatment indicator; only 20% get treated
    Y = Y1 * D + Y0 * (1 - D)  # observed Y
    data = pd.DataFrame({"Y": Y, "D": D, "Z": 1 + Z})  # we artificially add an intercept to the covariates
    return data
```

Analyze the RCT data with Precision Adjustment

Consider

- classical 2-sample approach, no adjustment (CL)
- classical linear regression adjustment (CRA)
- interactive regression adjusment (IRA)

Carry out inference using robust inference, using the sandwich formulas (Eicker-Huber-White).

Observe that CRA delivers estimates that are less efficient than CL (pointed out by Freedman), whereas IRA delivers estimates that are more efficient (pointed out by Lin). In order for CRA to be more efficient than CL, we need the linear model to be a correct model of the conditional expectation function of Y given D and X, which is not the case here.

```
data = gen_data(123)
```

```
CL = smf.ols("Y ~ D", data=data).fit()
# we are interested in the coefficients on variable "D".
CL.get_robustcov_results(cov_type="HC0").summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.003
Model:	OLS	**Adj. R-squared:**	0.002
Method:	Least Squares	**F-statistic:**	3.460
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	0.0632
Time:	15:31:32	**Log-Likelihood:**	-1424.8
No. Observations:	1000	**AIC:**	2854.
Df Residuals:	998	**BIC:**	2863.
Df Model:	1		
Covariance Type:	HC0		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0174	0.035	0.491	0.624	-0.052	0.087
D	-0.1499	0.081	-1.860	0.063	-0.308	0.008

Omnibus:	0.033	**Durbin-Watson:**	2.121
Prob(Omnibus):	0.984	**Jarque-Bera (JB):**	0.005
Skew:	-0.002	**Prob(JB):**	0.998
Kurtosis:	3.010	**Cond. No.**	2.67

Notes:
[1] Standard Errors are heteroscedasticity robust (HC0)

```
CRA = smf.ols("Y ~ D + Z", data=data).fit()      # classical
CRA.get_robustcov_results(cov_type="HC0").summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.392
Model:	OLS	**Adj. R-squared:**	0.391
Method:	Least Squares	**F-statistic:**	117.4
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	1.59e-46
Time:	15:31:36	**Log-Likelihood:**	-1177.7
No. Observations:	1000	**AIC:**	2361.
Df Residuals:	997	**BIC:**	2376.
Df Model:	2		
Covariance Type:	HC0		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.6344	0.043	14.854	0.000	0.551	0.718
D	-0.2229	0.119	-1.873	0.061	-0.457	0.011
Z	-0.6282	0.041	-15.236	0.000	-0.709	-0.547

Omnibus:	139.846	**Durbin-Watson:**	2.099
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1646.612
Skew:	-0.072	**Prob(JB):**	0.00
Kurtosis:	9.285	**Cond. No.**	4.22

Notes:
[1] Standard Errors are heteroscedasticity robust (HC0)

```
# if we demean the covariates, then the intercept can be interpreted
# as an estimate of the expected outcome under control
data['Zdemean'] = data['Z'] - data['Z'].mean(axis=0)
CRA = smf.ols("Y ~ D + Zdemean", data=data).fit()
CRA.get_robustcov_results(cov_type="HC0").summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.392
Model:	OLS	**Adj. R-squared:**	0.391
Method:	Least Squares	**F-statistic:**	117.4
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	1.59e-46
Time:	15:31:38	**Log-Likelihood:**	-1177.7
No. Observations:	1000	**AIC:**	2361.
Df Residuals:	997	**BIC:**	2376.
Df Model:	2		
Covariance Type:	HC0		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0311	0.014	2.274	0.023	0.004	0.058
D	-0.2229	0.119	-1.873	0.061	-0.457	0.011
Zdemean	-0.6282	0.041	-15.236	0.000	-0.709	-0.547

Omnibus:	139.846	**Durbin-Watson:**	2.099
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1646.612
Skew:	-0.072	**Prob(JB):**	0.00
Kurtosis:	9.285	**Cond. No.**	2.67

Notes:
[1] Standard Errors are heteroscedasticity robust (HC0)

```
# However, then we need to correct the standard error associated
# with the intercept, to account for the variance in estimating the means.
# The standard error for D does not need any correction
J = np.mean(1 - data['D'])
# the HC0 standard error for the intercept is the second moment of the following score quantity
score = CRA.resid * (1 - data['D']) / J
# however, now we need to add a correction to the score to account for the
# error in the means
score += data[['Zdemean']] @ CRA.params[['Zdemean']]
print(f"Corrected stderr['Intercept']: {np.sqrt(np.mean(score**2) / len(data)):.4f}")
```

```
Corrected stderr['Intercept']: 0.0325
```

```
# for the interactive approach, we need to demean the covariates Z to interpret
# the coefficient of D as the ATE
IRA = smf.ols("Y ~ D + Zdemean + Zdemean*D", data=data).fit()  # interactive approach
IRA.get_robustcov_results(cov_type="HC1").summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.991
Model:	OLS	**Adj. R-squared:**	0.991
Method:	Least Squares	**F-statistic:**	3.773e+04
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	0.00
Time:	15:31:46	**Log-Likelihood:**	928.72
No. Observations:	1000	**AIC:**	-1849.
Df Residuals:	996	**BIC:**	-1830.
Df Model:	3		
Covariance Type:	HC1		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0394	0.003	11.797	0.000	0.033	0.046
D	-0.0790	0.008	-9.930	0.000	-0.095	-0.063
Zdemean	-1.0059	0.003	-301.249	0.000	-1.012	-0.999
Zdemean:D	1.9884	0.008	264.720	0.000	1.974	2.003

Omnibus:	1.179	**Durbin-Watson:**	2.047
Prob(Omnibus):	0.555	**Jarque-Bera (JB):**	1.052
Skew:	-0.066	**Prob(JB):**	0.591
Kurtosis:	3.087	**Cond. No.**	2.76

Notes:
[1] Standard Errors are heteroscedasticity robust (HC1)

```
Start coding or generate with AI.
```

```
# However, in the interactive approach we also need to correct
# the standard error associated with D, to account for the estimation of the means
correction = np.var(data[['Zdemean']].values @ IRA.params[['Zdemean:D']]) / len(data)
print(f"Corrected stderr['D']: {np.sqrt(IRA.HC0_se['D']**2 + correction):.4f}")
```

```
Corrected stderr['D']: 0.0634
```

```
# And as before we need to correct the standard error associated
# with the intercept, to account for the variance in estimating the means.
J = np.mean(1 - data['D'])
score = (IRA.resid * (1 - data['D']) + J * data[['Zdemean']] @ IRA.params[['Zdemean']]) / J
print(f"Corrected stderr['Intercept']: {np.sqrt(np.mean(score**2) / len(data)):.4f}")
```

```
Corrected stderr['Intercept']: 0.0320
```

Using classical standard errors (non-robust) is misleading here.

We don't teach non-robust standard errors in econometrics courses, but the default statistical inference for the `fit` procedure in python, `smf.ols()`, still uses 100 year old concepts, perhaps in part due to historical legacy.

Here the non-robust standard errors suggest that there is not much difference between the different approaches, contrary to the conclusions reached using the robust standard errors.

```
smf.ols("Y ~ D", data).fit().summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.003
Model:	OLS	**Adj. R-squared:**	0.002
Method:	Least Squares	**F-statistic:**	3.382
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	0.0662
Time:	15:31:54	**Log-Likelihood:**	-1424.8
No. Observations:	1000	**AIC:**	2854.
Df Residuals:	998	**BIC:**	2863.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0174	0.035	0.492	0.623	-0.052	0.087
D	-0.1499	0.081	-1.839	0.066	-0.310	0.010

Omnibus:	0.033	**Durbin-Watson:**	2.121
Prob(Omnibus):	0.984	**Jarque-Bera (JB):**	0.005
Skew:	-0.002	**Prob(JB):**	0.998
Kurtosis:	3.010	**Cond. No.**	2.67

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
smf.ols("Y ~ D + Z", data).fit().summary()
```

OLS Regression Results

Dep. Variable:	Y	**R-squared:**	0.392
Model:	OLS	**Adj. R-squared:**	0.391
Method:	Least Squares	**F-statistic:**	321.2
Date:	Sun, 03 May 2026	**Prob (F-statistic):**	2.07e-108
Time:	15:31:59	**Log-Likelihood:**	-1177.7
No. Observations:	1000	**AIC:**	2361.
Df Residuals:	997	**BIC:**	2376.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.6344	0.037	17.201	0.000	0.562	0.707
D	-0.2229	0.064	-3.497	0.000	-0.348	-0.098
Z	-0.6282	0.025	-25.238	0.000	-0.677	-0.579

Omnibus:	139.846	**Durbin-Watson:**	2.099
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1646.612
Skew:	-0.072	**Prob(JB):**	0.00
Kurtosis:	9.285	**Cond. No.**	4.22

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Verify Asymptotic Approximations Hold in Finite-Sample Simulation Experiment

```
from joblib import Parallel, delayed


def exp(it):
```

```
    data = gen_data(it)
    data['Zdemean'] = data['Z'] - data['Z'].mean(axis=0)
    CL = smf.ols("Y ~ D", data).fit()
    CLcoef = CL.params["D"]
    CLint = CL.params["Intercept"]
    CRA = smf.ols("Y ~ D + Zdemean", data).fit()
    CRAcoef = CRA.params["D"]
    CRAint = CRA.params["Intercept"]
    IRA = smf.ols("Y ~ D + Zdemean+ Zdemean*D", data).fit()
    IRAcoef = IRA.params["D"]
    IRAint = IRA.params["Intercept"]
    return CLcoef, CLint, CRAcoef, CRAint, IRAcoef, IRAint


B = 1000
res = Parallel(n_jobs=-1, verbose=3)(delayed(exp)(it) for it in range(B))
```

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done  38 tasks      | elapsed:    3.1s
[Parallel(n_jobs=-1)]: Done 798 tasks      | elapsed:   16.6s
[Parallel(n_jobs=-1)]: Done 1000 out of 1000 | elapsed:   19.8s finished
```

```
CLcoefs, CLints, CRAcoefs, CRAints, IRAcoefs, IRAints = map(lambda x: np.array(x), zip(*res))
```